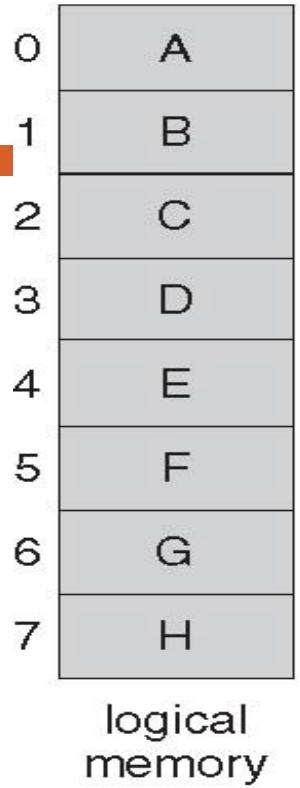
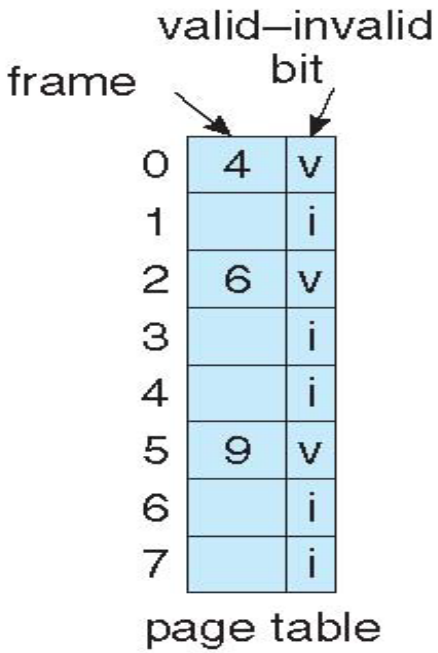


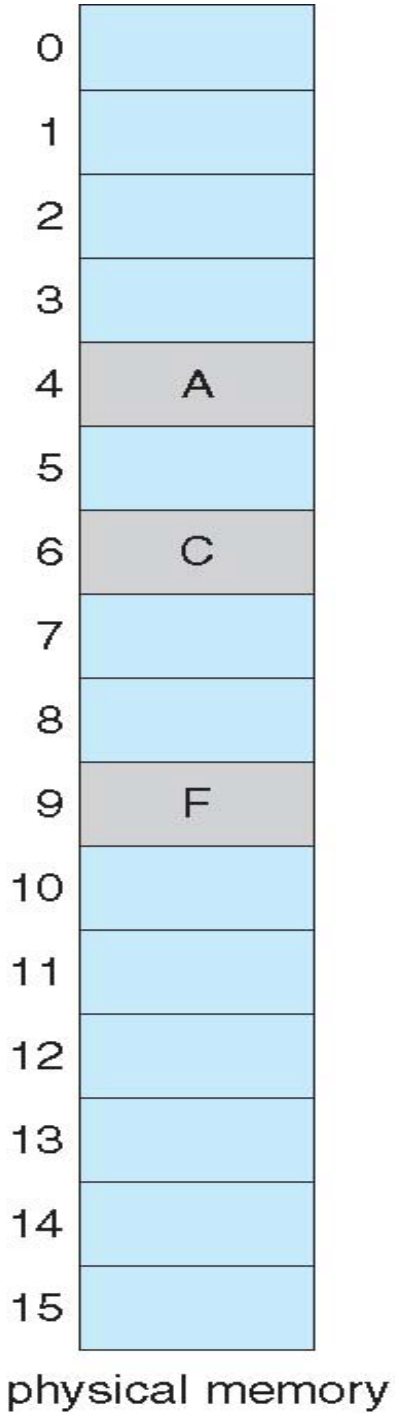
Demand Paging

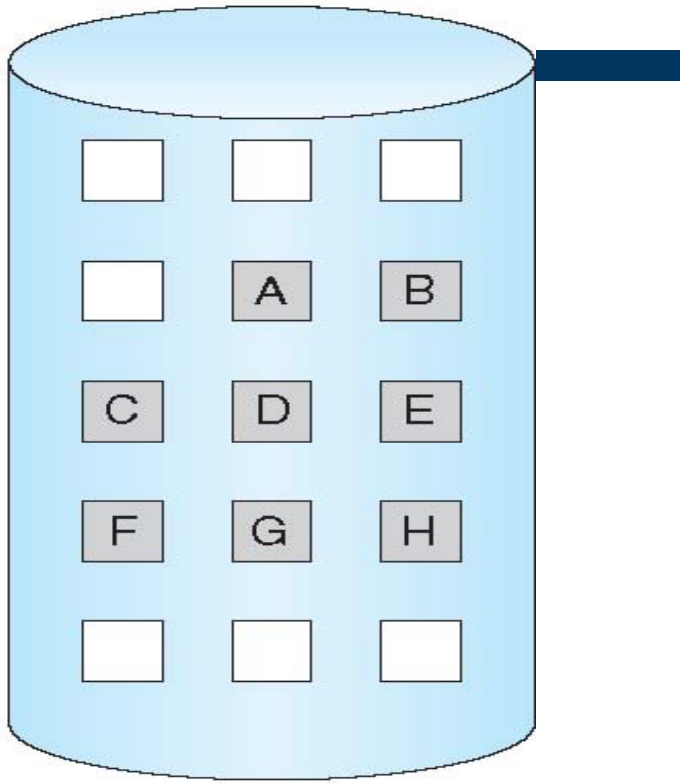
Demand Paging

- Pages only brought into memory when referenced
  - Only *code & data* that is needed by a process needs to be loaded.
  - So, called: ***demand paging***.
  - Essential for time and space efficiency!
- *Few systems try to anticipate future needs.*

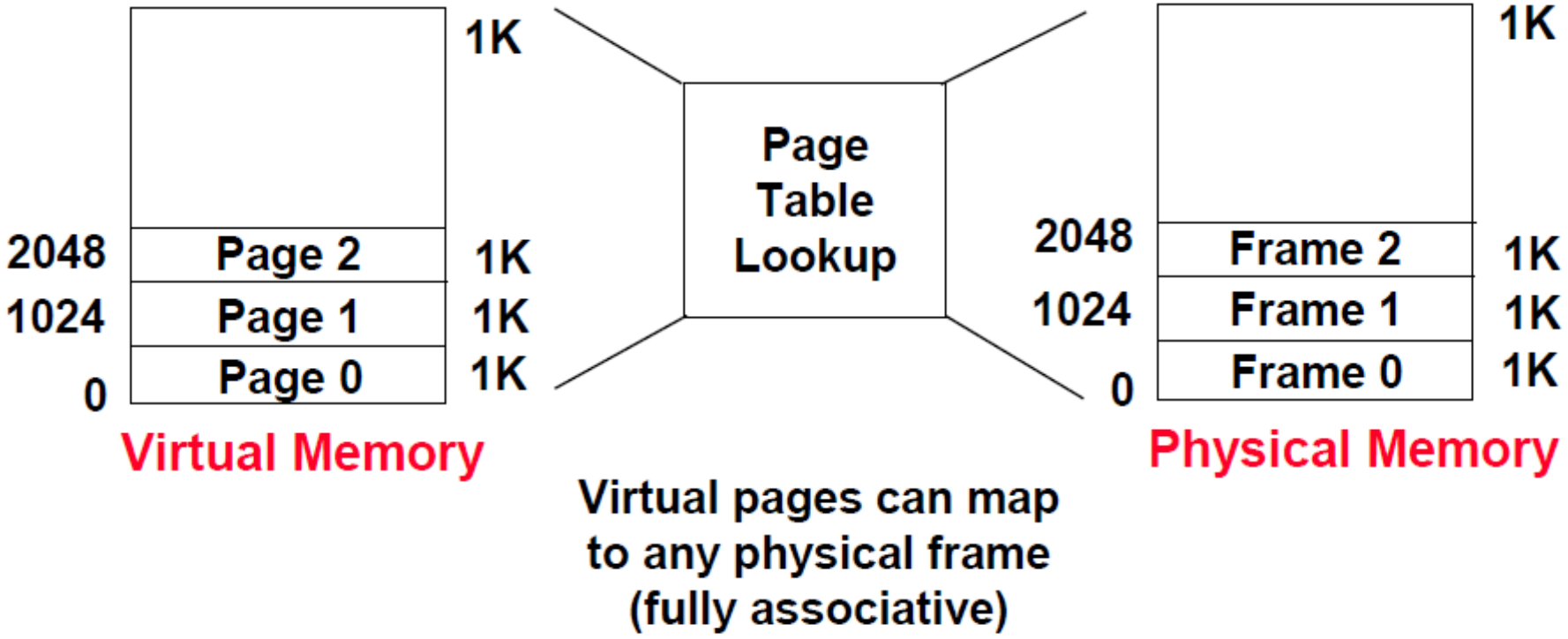

0 A
1 B
2 C
3 D
4 E
5 F
6 G
7 H
logical memory


valid–invalid bit
frame
0 4 v
1 i
2 6 v
3 i
4 i
5 9 v
6 i
7 i
page table

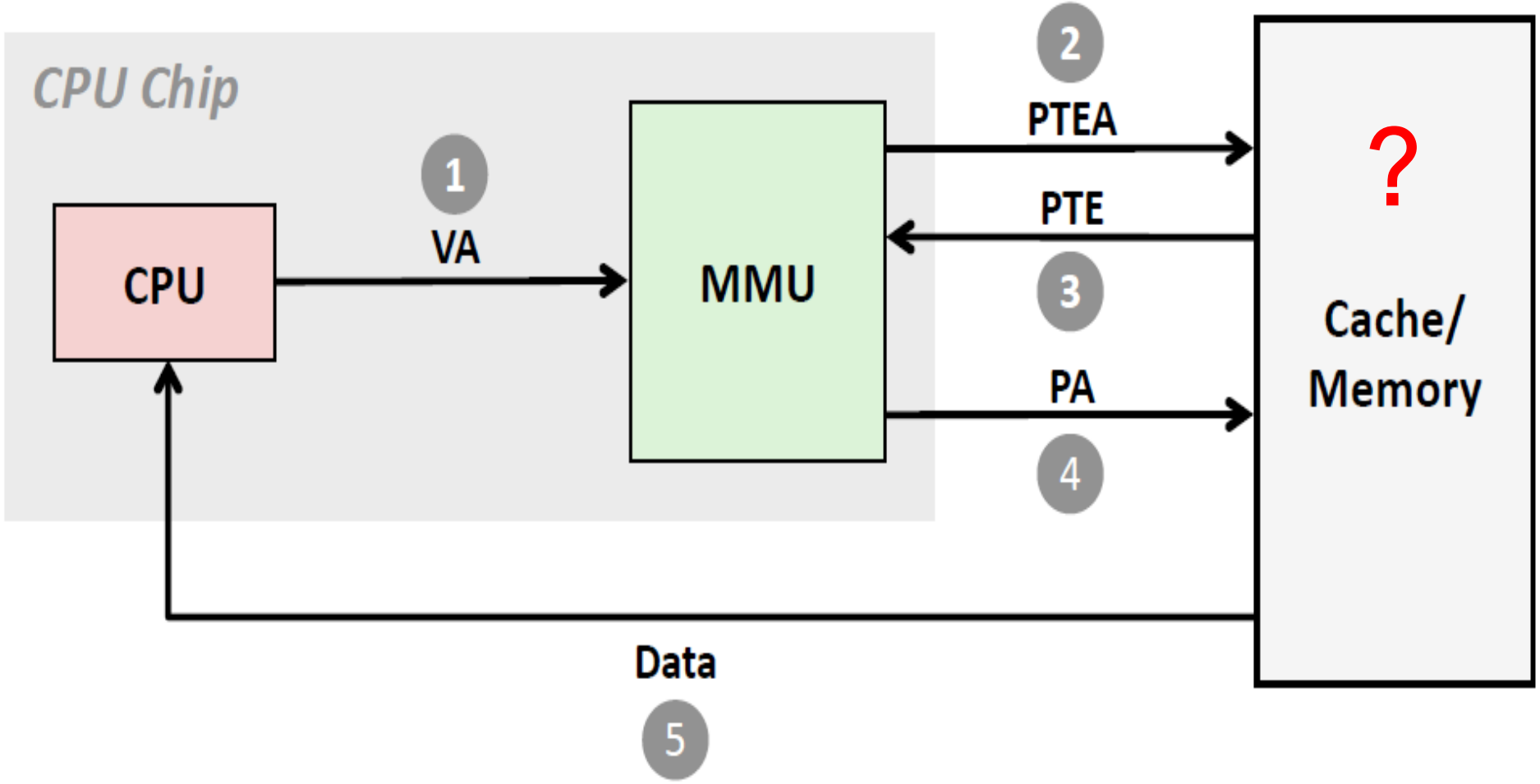

0
1
2
3
4 A
5
6 C
7
8
9 F
10
11
12
13
14
15
physical memory


A B
C D E
F G H

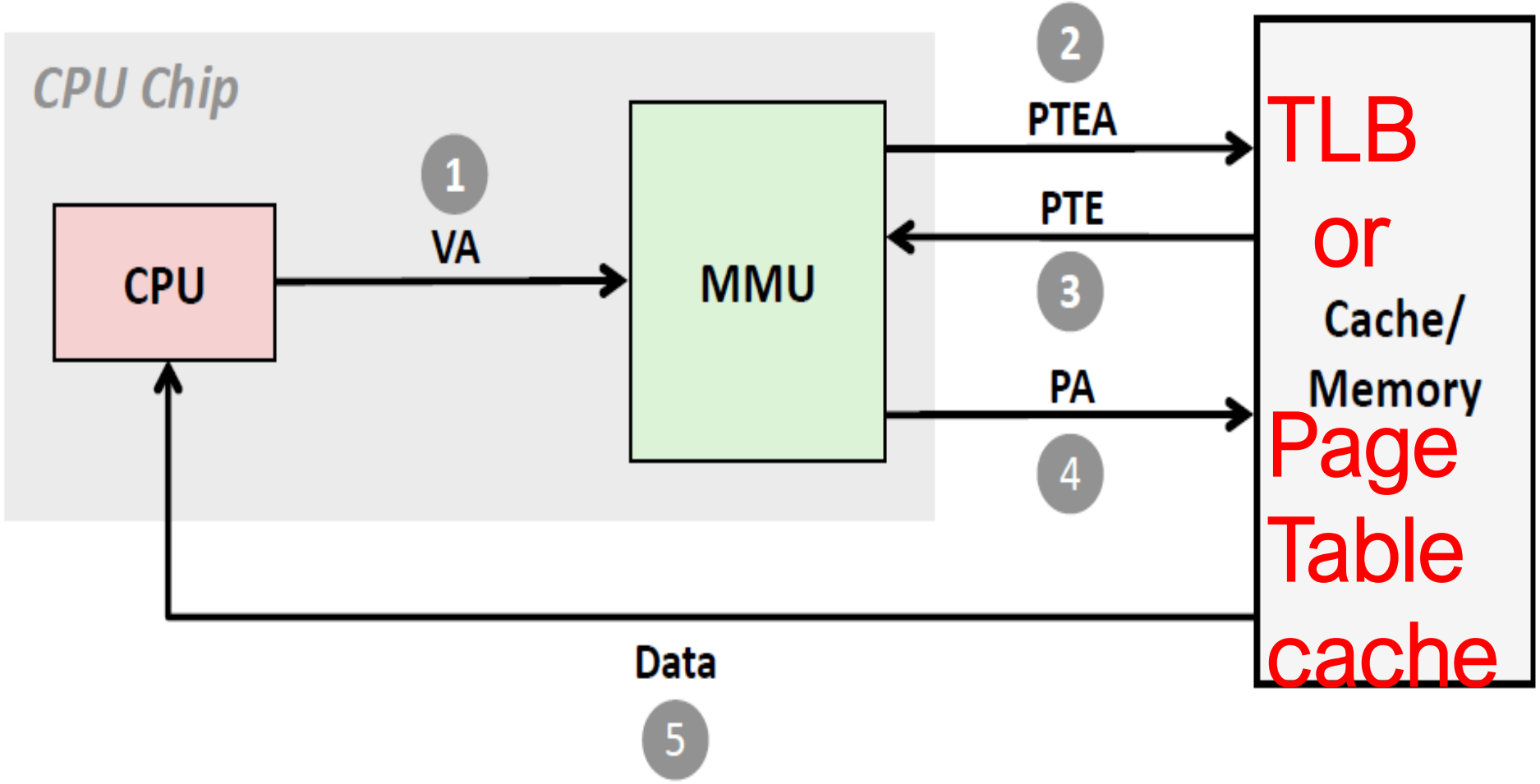
Page Address Translation



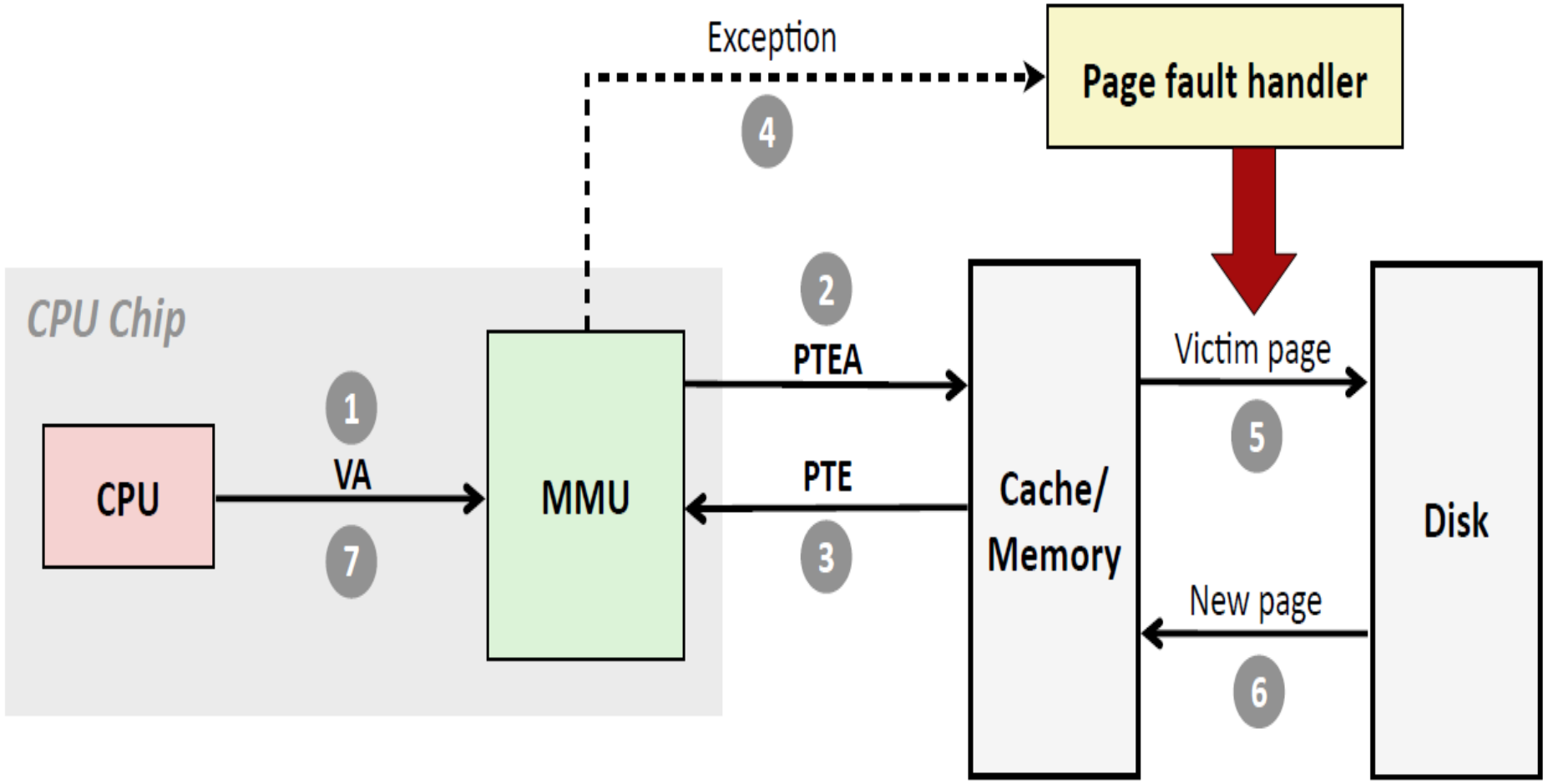
Address Translation: Page Hit!



Address Translation: Page Hit!



Address Translation: Page Fault



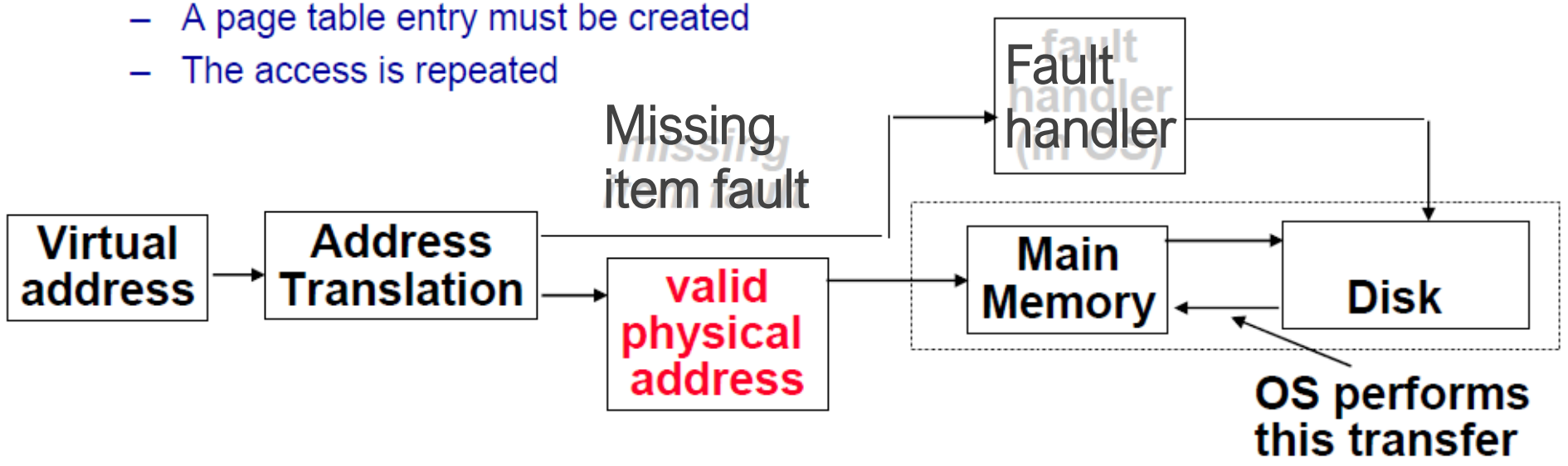
Handling Page Faults

1. CPU requests ***Virtual Address*** (VA) from MMU.
2. Request ***Page Table Entry Address*** (PTEA).
3. MMU tries **PTE from page table** in cache (memory).
4. **Valid Bit = 0 :** MMU triggers page fault exception (trap!).
5. Page fault handler **gets victim page** – swaps out to disk.
6. Handler **swaps in new page** – updates PTE.
7. Return to original process – restarts faulting instruction.

Page Fault!

- Programs use virtual address which may include pages that currently reside in secondary storage.

- **Page Fault:** *If an entry is not found then address is not currently located in physical memory.*
  - A page table entry must be created
  - The access is repeated



What is a Page Fault?

- User process references page with invalid PTE?
  - While pulling pages off disk for one process, OS runs another process from ready queue.
  - Suspended process sits on wait queue.
  - Page fault is a type of ….?

Page Fault → System Call

- User process references page with invalid PTE?
  - Page fault is a type of system call!
  - Memory Management Unit (MMU) traps to OS.

What causes Page Faults?

- **Compulsory Faults:** Pages *currently not in physical (main) memory* – i.e. swapped out.
- **Capacity Faults:** *Not enough main memory* – Must somehow increase available memory size.
- **Policy Faults:** Pages were in memory – but kicked out because of *page replacement algorithm*.

Page Fault Handling

- **How do we find pages (not in main memory)?**
  - Check: *Page Table Base Register* (PTBR).
  - Check: *Translation Lookaside Buffer* (TLB).
  - Otherwise: Must swap page in from secondary storage (HDD).

Page Fault: What now?

- **Address translation: page not in main memory:**
  - Locate page: TLB (cache) or disk (secondary storage).
  - Find an available physical frame (or evict a page).
  - Write the contents (page) into the frame.
  - Update page table.
- **Pages fetched only on demand (when accessed).**

Page Replacement Algorithms

- Goal of any page replacement algorithm:
  - Reduce fault rate by removing “best” victim page.
  - Best page to evict is one never to be referenced again.
- Analysis of page replacement algorithms:
  - Assume that a process pages against itself.
  - Using a fixed number of page frames.

Evaluate Page Replacement Algorithms

- **Goal:** Want lowest page-fault rate on both first access and re-access.

- **Evaluate:** Run it on string of memory page references (reference string) and compute number of page faults on that string.

  - String is just page numbers – not full addresses.

  - Repeated access to page X – no page fault.

Page Replacement Algorithms

- OPT (Optimal).
- NRU (Not Recently Used).
- FIFO (First In First Out).
- Second Chance.
- Clock.
- LRU (Least Recently Used).
- Aging.

Page Replacement: Random

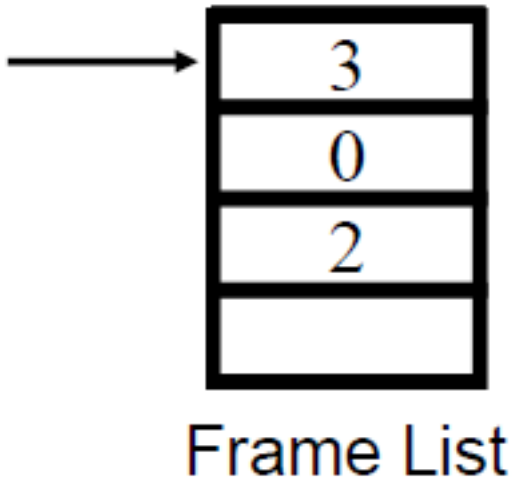
- Pick random page for every replacement.
- Problems?

Page Replacement: Random

- Pick random page for every replacement.
- Unpredictable – hard to make real-time guarantees about algorithm performance.

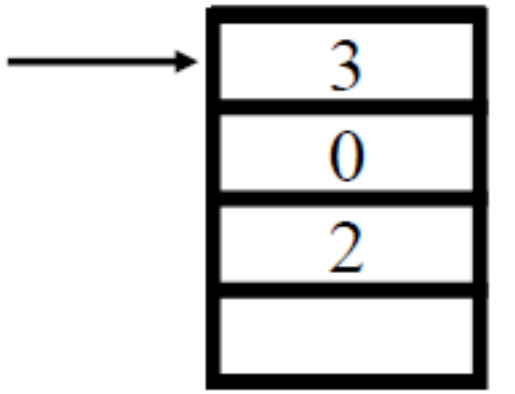
Page Replacement: FIFO

- **FIFO (First In, First Out):**
  - Throw out oldest page.
  - Simple to implement – Single pointer to frame-list.
  - **Good:** ?
  - **Bad:** ?



Page Replacement: FIFO

- **FIFO (First In, First Out):**
  - Throw out oldest page.
  - Simple to implement – Single pointer to frame-list.
  - **Fair:** Let every page live in memory for same amount of time.
  - **Bad:** Throws out heavily used pages instead of infrequently used.



Frame List

FIFO Algorithm

- Reference string (N = 20):

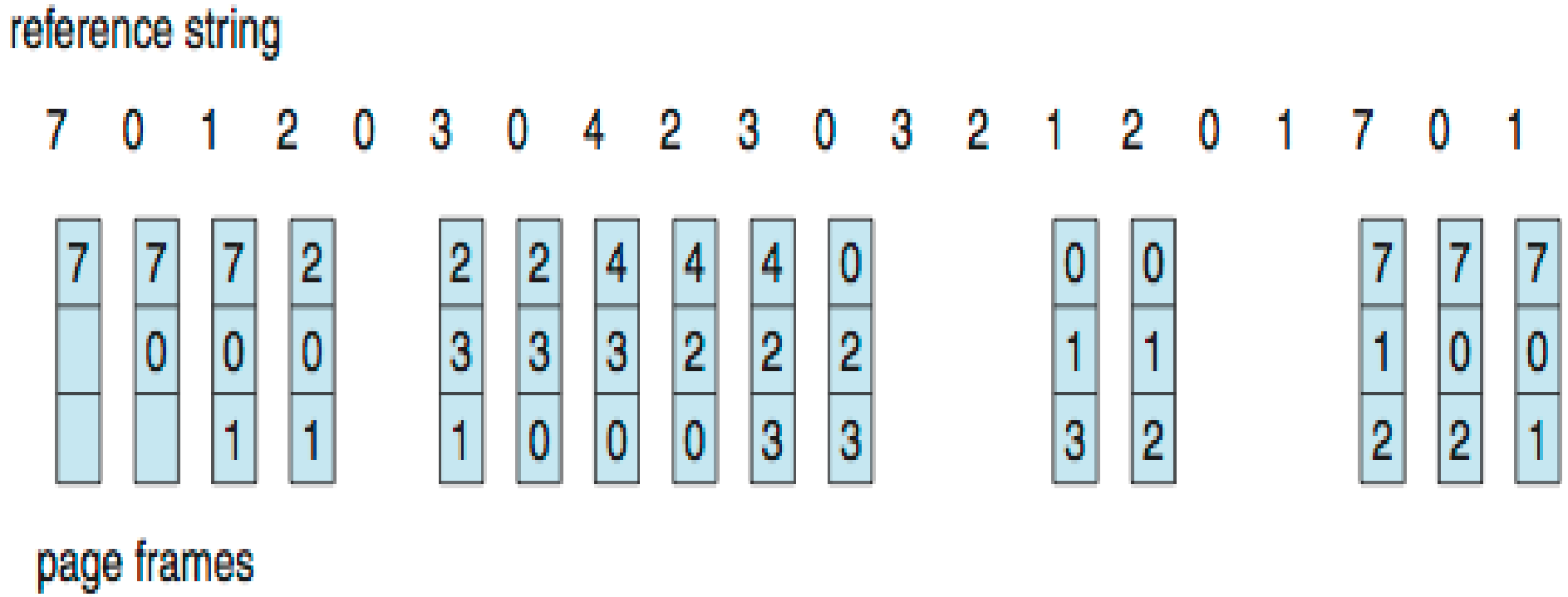
7, 0 , 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

- **Frames = 3**

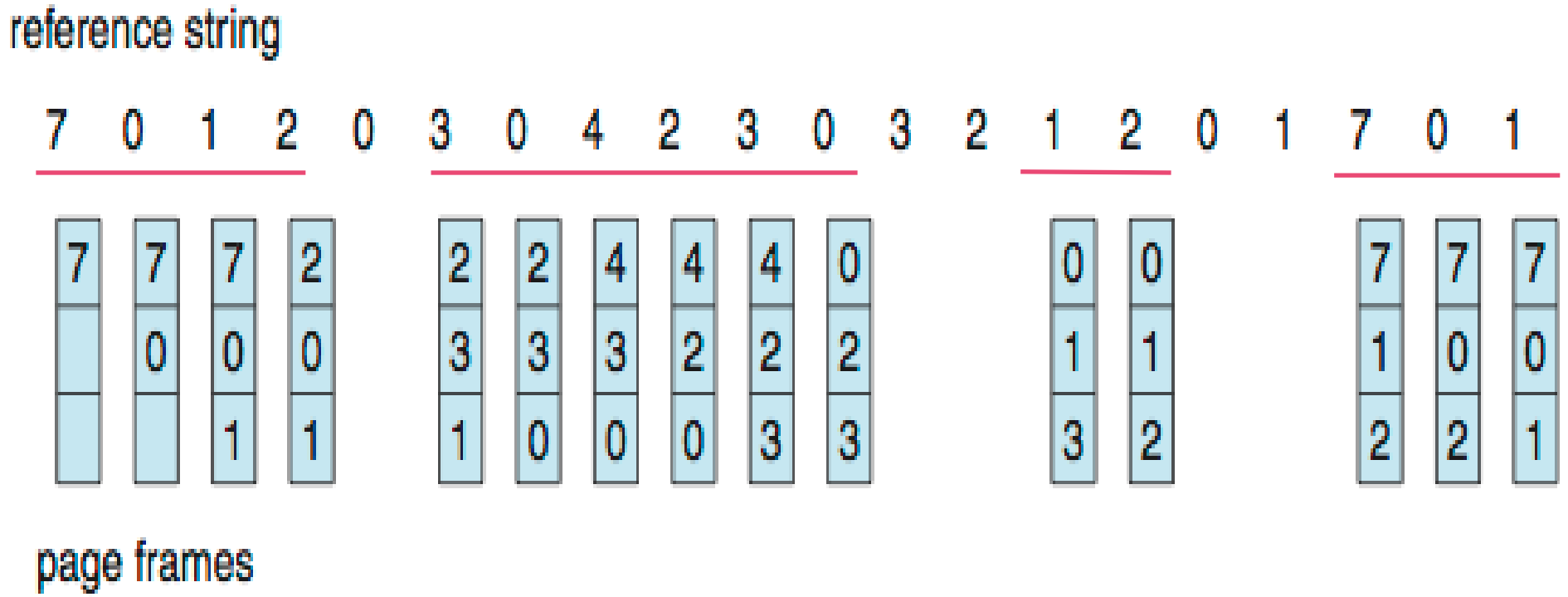
- **i.e.:** 3 pages can be in memory at a time (per process).

- *How many page faults?*

FIFO Algorithm: 15 Page Faults



FIFO Algorithm: 15 Page Faults

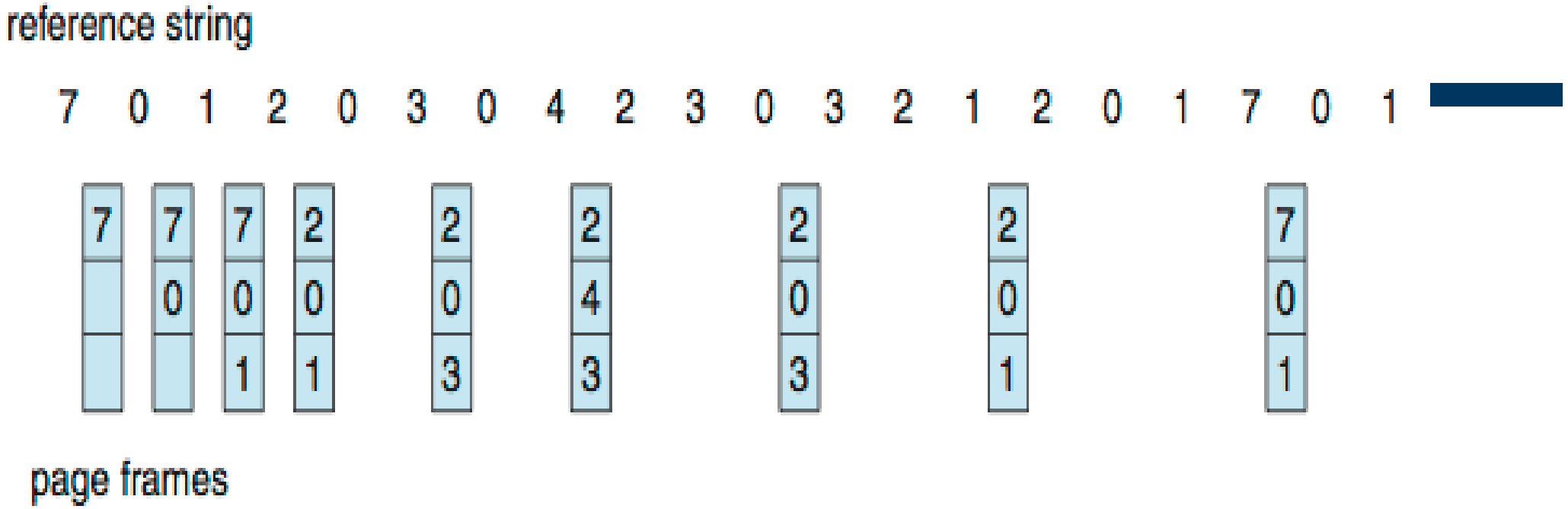


OPT (Optimal) Page Replacement

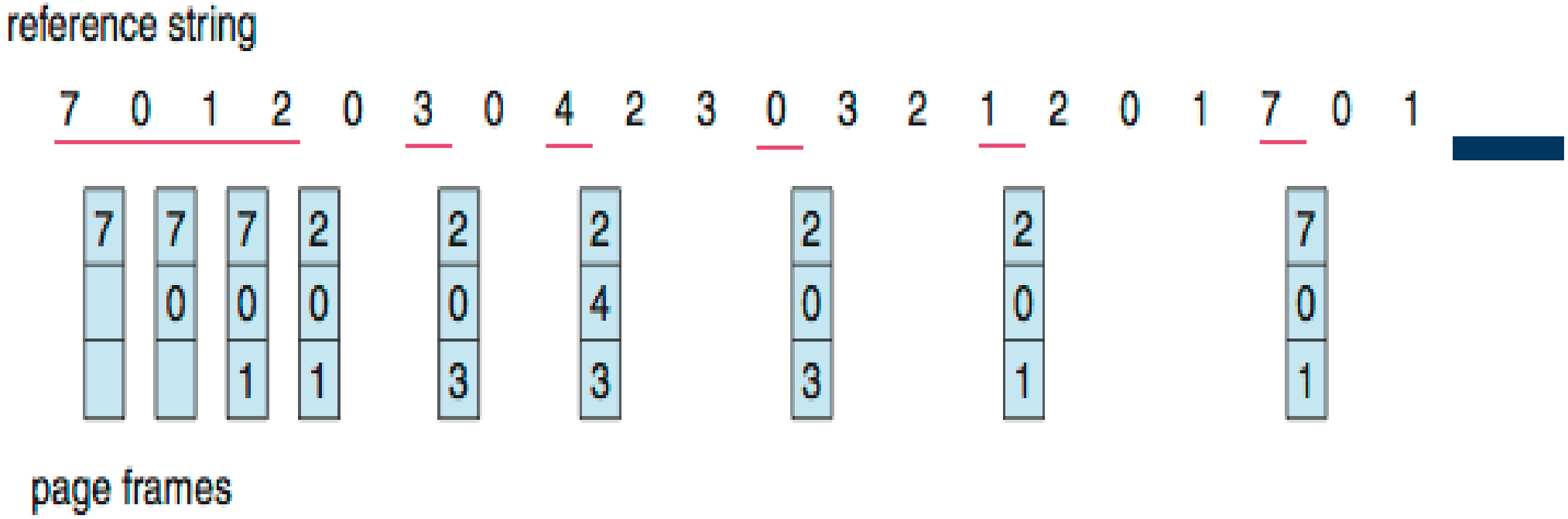
- Replace the page that won't be used for the longest time.
- Problems …?

OPT (Optimal) Page Replacement

- Replace the page that won't be used for the longest time.
- We cannot know what N pages will be next – but we can generate reference string test-cases.
- Used as "bench-mark" for measuring how well other page replacement algorithms work.



- **How many page faults?**



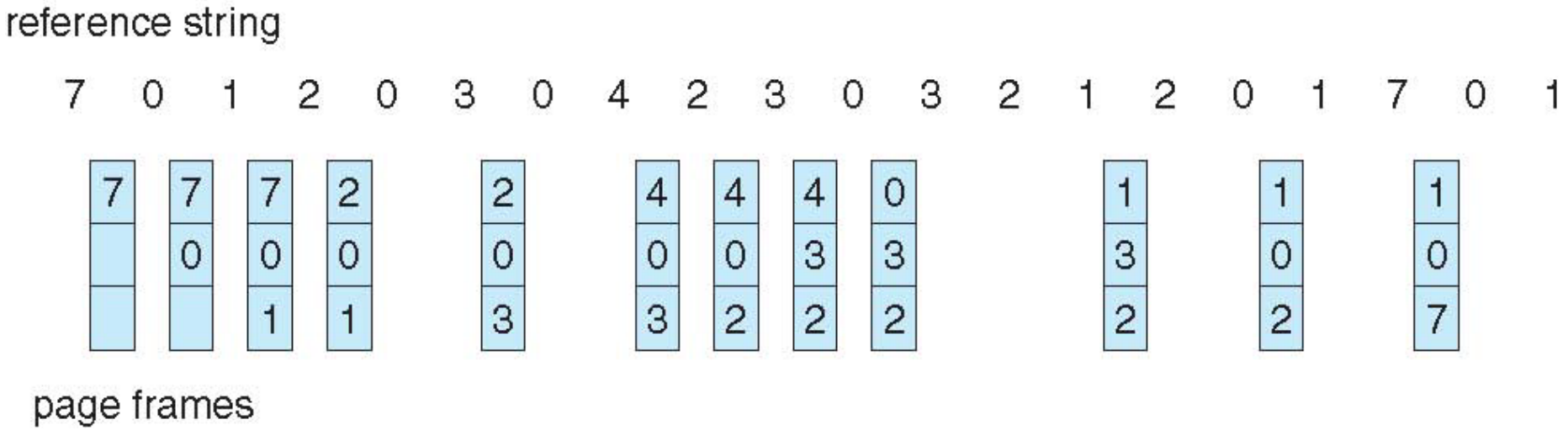
- **e.g.:** First replace “**7**”, then “**1**”, then “**0**”, …
- **Optimal case:** 9 page faults (N = 20).

Least Recently Used (LRU)

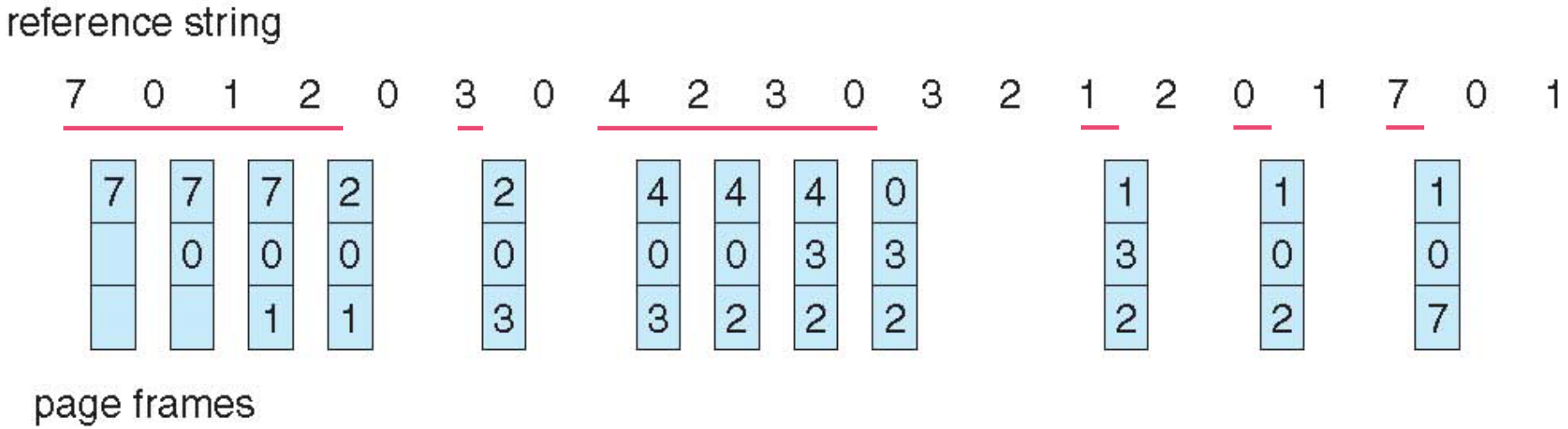
- Use past knowledge – rather than future.
- Replace page not been used for longest time – (FIFO replaces pages regardless of usage).
- Associate time of use with each page.

Least Recently Used (LRU)

- **How many page faults?**



Least Recently Used (LRU)

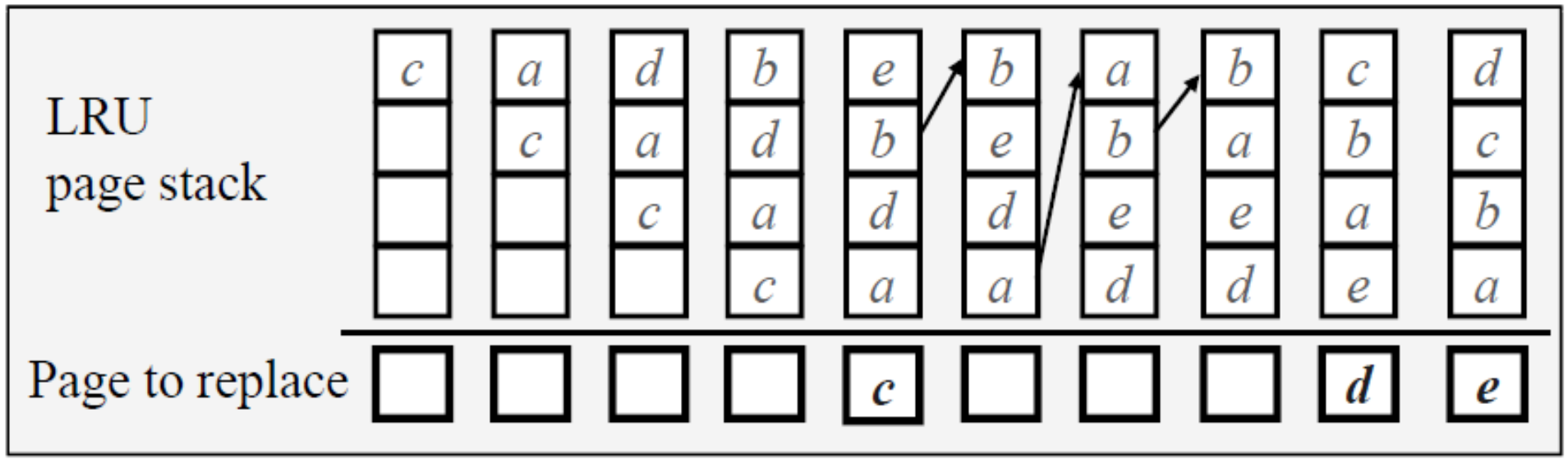


- **12 faults** – Better than FIFO – worse than OPT.
- How to implement LRU?

LRU Counter Implementation

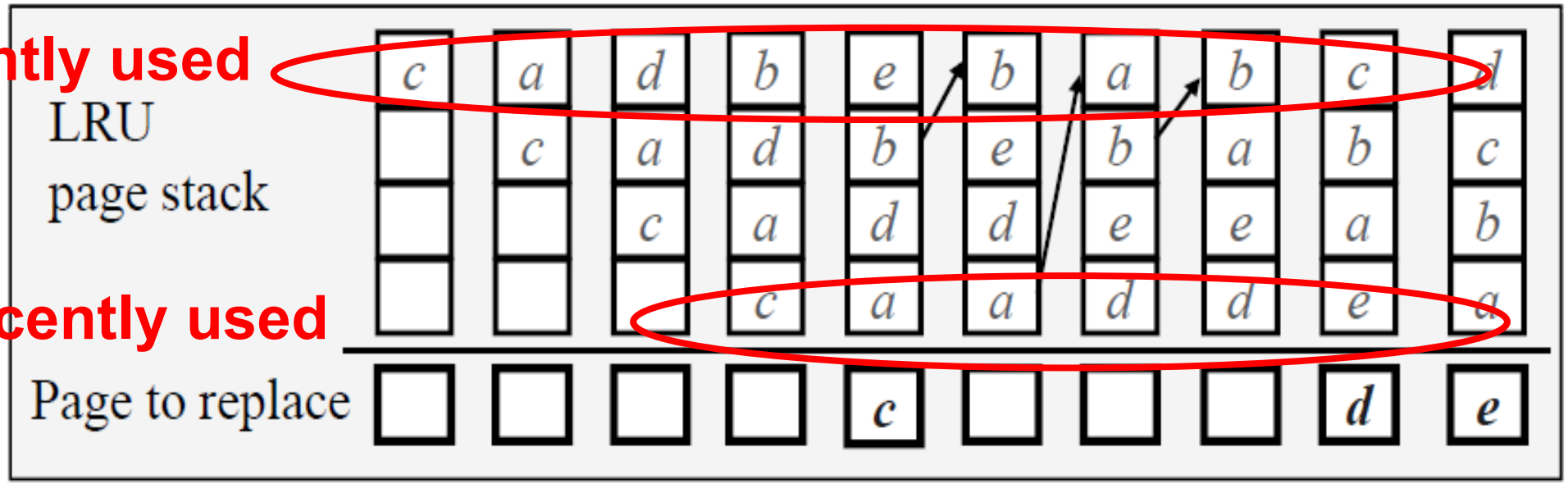
- Every page has a counter.
- Every time a page is referenced, counter is incremented (current time copied in).
- We replace the page with the smallest time value.
- What data structure to use here?

Time		0	1	2	3	4	5	6	7	8	9	10
Requests			*c*	*a*	*d*	*b*	*e*	*b*	*a*	*b*	*c*	*d*
Page Frames	0	*a*	*a*	*a*	*a*	*a*	*a*	*a*	*a*	*a*	*a*	*a*
	1	*b*	*b*	*b*	*b*	*b*	*b*	*b*	*b*	*b*	*b*	*b*
	2	*c*	*c*	*c*	*c*	*c*	***e***	*e*	*e*	*e*	*e*	***d***
	3	*d*	*d*	*d*	*d*	*d*	*d*	*d*	*d*	*d*	***c***	*c*
Faults							•				•	•


LRU
page stack
Page to replace
c a d b e b a b c d
c a d b e b a b c d
a d b e b a b c d
c a d b e b a b c d
c a d b e e e e e e
c a d b d d d d d d
c a d b a a a a a a
c a d b c c c c c c
c a d b e b a b c d
c a d b e b a b c d
c a d b e b a b c d

Time		0	1	2	3	4	5	6	7	8	9	10
Requests			*c*	*a*	*d*	*b*	*e*	*b*	*a*	*b*	*c*	*d*
Page Frames	0	*a*	*a*	*a*	*a*	*a*	*a*	*a*	*a*	*a*	*a*	*a*
	1	*b*	*b*	*b*	*b*	*b*	*b*	*b*	*b*	*b*	*b*	*b*
	2	*c*	*c*	*c*	*c*	*c*	***e***	*e*	*e*	*e*	*e*	***d***
	3	*d*	*d*	*d*	*d*	*d*	*d*	*d*	*d*	*d*	***c***	*c*
Faults							•				•	•

Most recently used



Least recently used

Most Frequently Used (MFU)

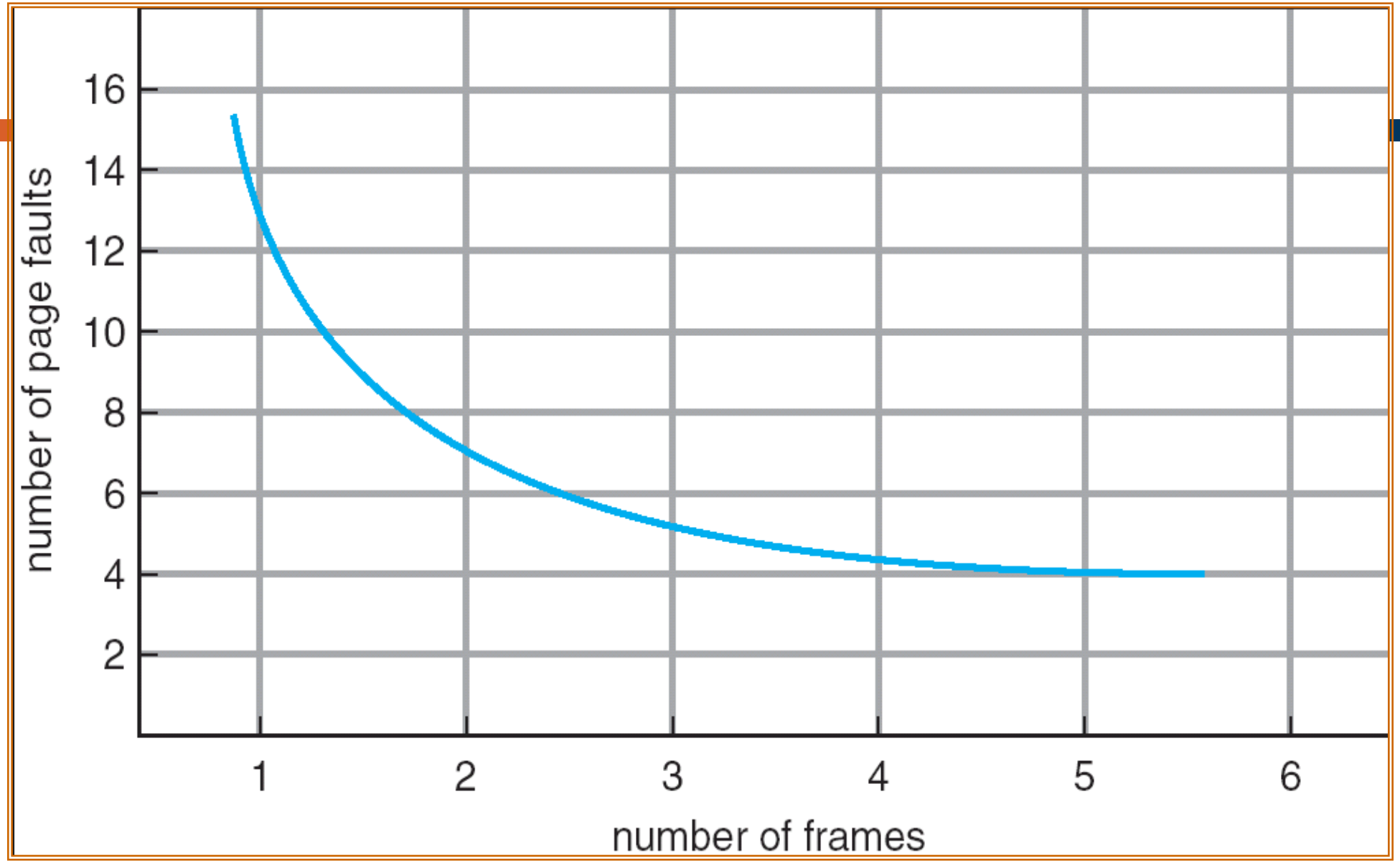
- **MFU Algorithm:**
  - Replaces *most frequently used* page.
  - *Rationale*: MFU page has been used a lot already and will not be needed immediately.
- Problems with LRU and MFU ?

Counting Algorithms

- All implementations and approximations of LRU & MFU need some hardware assistance (counter).
- Keep a time stamp for each page with time of last access. Throw out the LRU/MRU page.
- OS must record time stamp for each memory access and to throw out a page must look at all pages.
- *Computationally Expensive.*

Page Replacement Algorithms

- Pros and cons of various page replacement algorithms aside – is there an easy way to reduce page faults for all page replacement algorithms?


number of page faults
16
14
12
10
8
6
4
2
1
2
3
4
5
6
number of frames

Page Replacement Algorithms

Belady's theorem:

Evicting the page that won't be used for the longest period minimises the page fault rate.

Page Replacement Algorithms

Belady's theorem:

Evicting the page that won't be used for the longest period minimises the page fault rate.

This is the OPT algorithm!

FIFO: Frames vs Page Faults

- More frames allocated to process – fewer page faults?
- Consider the reference string:

  3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4

- FIFO: How many page faults if we have ***frames = 3*** ?

Page requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest page	3	2	1	0	3	2	4	4	4	1	0	0
		3	2	1	0	3	2	2	2	4	1	1
Oldest page			3	2	1	0	3	3	3	2	4	4

9 page faults

FIFO: Frames vs Page Faults

- More frames allocated to process – fewer page faults?
- Consider the reference string:

  3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4

- FIFO: How many page faults if we have ***frames = 4*** ?

10 page faults

Page requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest page	3	2	1	0	0	0	4	3	2	1	0	4
		3	2	1	1	1	0	4	3	2	1	0
			3	2	2	2	1	0	4	3	2	1
Oldest page				3	3	3	2	1	0	4	3	2

Page requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest page	3	2	1	0	3	2	4	4	4	1	0	0
		3	2	1	0	3	2	2	2	4	1	1
Oldest page			3	2	1	0	3	3	3	2	4	4

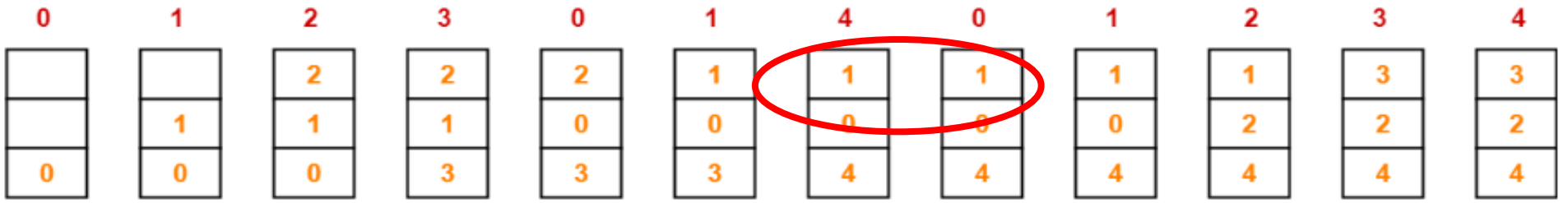
Page requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest page	3	2	1	0	0	0	4	3	2	1	0	4
		3	2	1	1	1	0	4	3	2	1	0
			3	2	2	2	1	0	4	3	2	1
Oldest page				3	3	3	2	1	0	4	3	2

Page requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest page	3	2	1	0	3	2	4	4	4	1	0	0
		3	2	1	0	3	2	2	2	4	1	1
Oldest page			3	2	1	0	3	3	3	2	4	4

Page requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest page	3	2	1	0	0	0	4	3	2	1	0	4
		3	2	1	1	1	0	4	3	2	1	0
			3	2	2	2	1	0	4	3	2	1
Oldest page				3	3	3	2	1	0	4	3	2

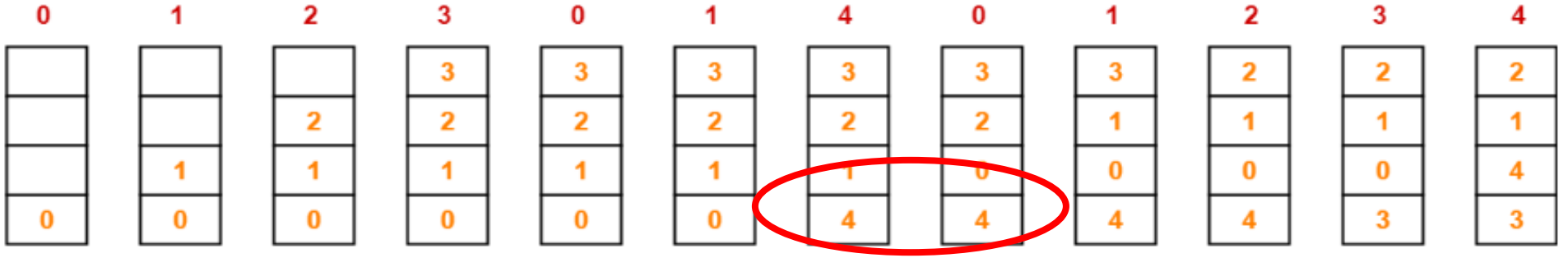
FIFO

Case-01: When frame size = 3



Number of page faults = 9

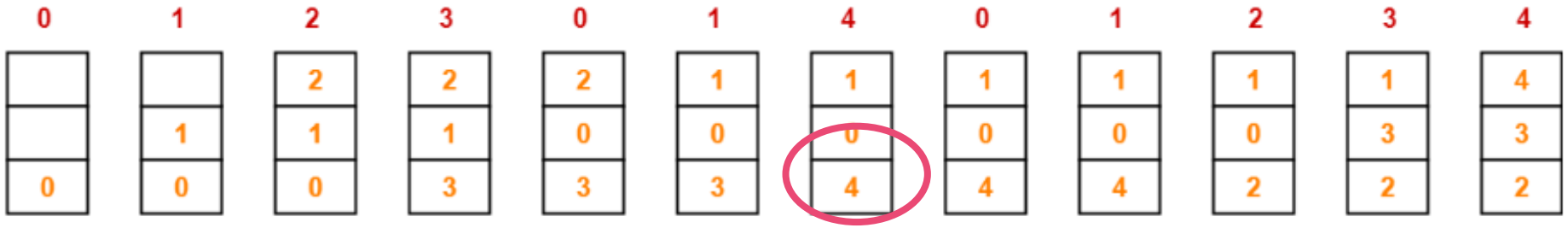
Case-02: When frame size = 4



Number of page faults = 10

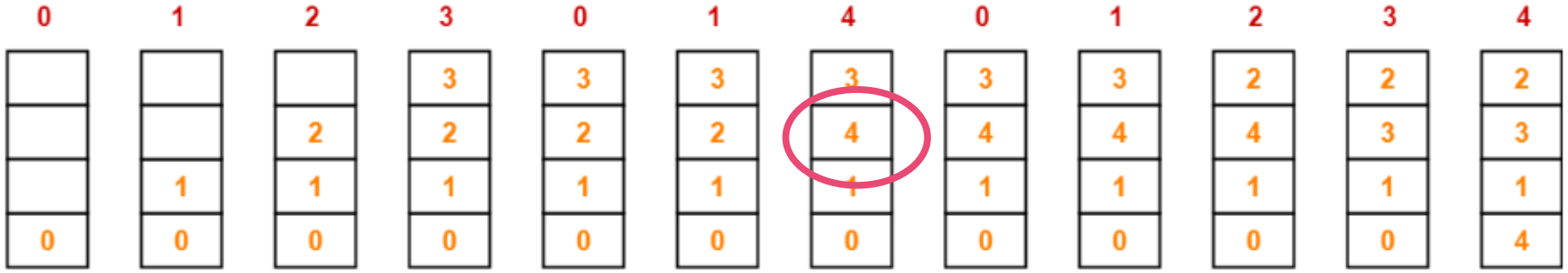
LRU

Case-01: When frame size = 3

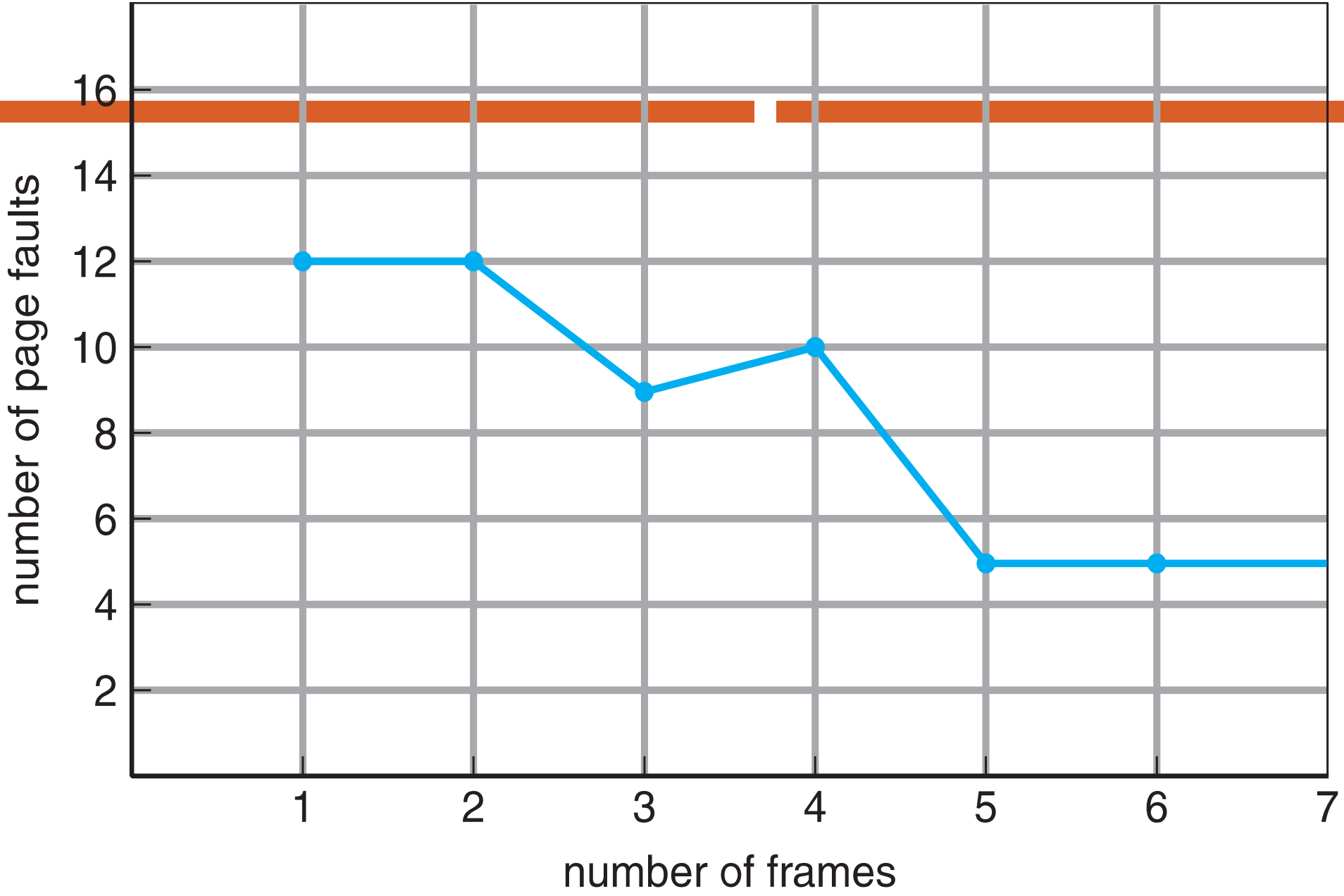


Number of page faults = 10

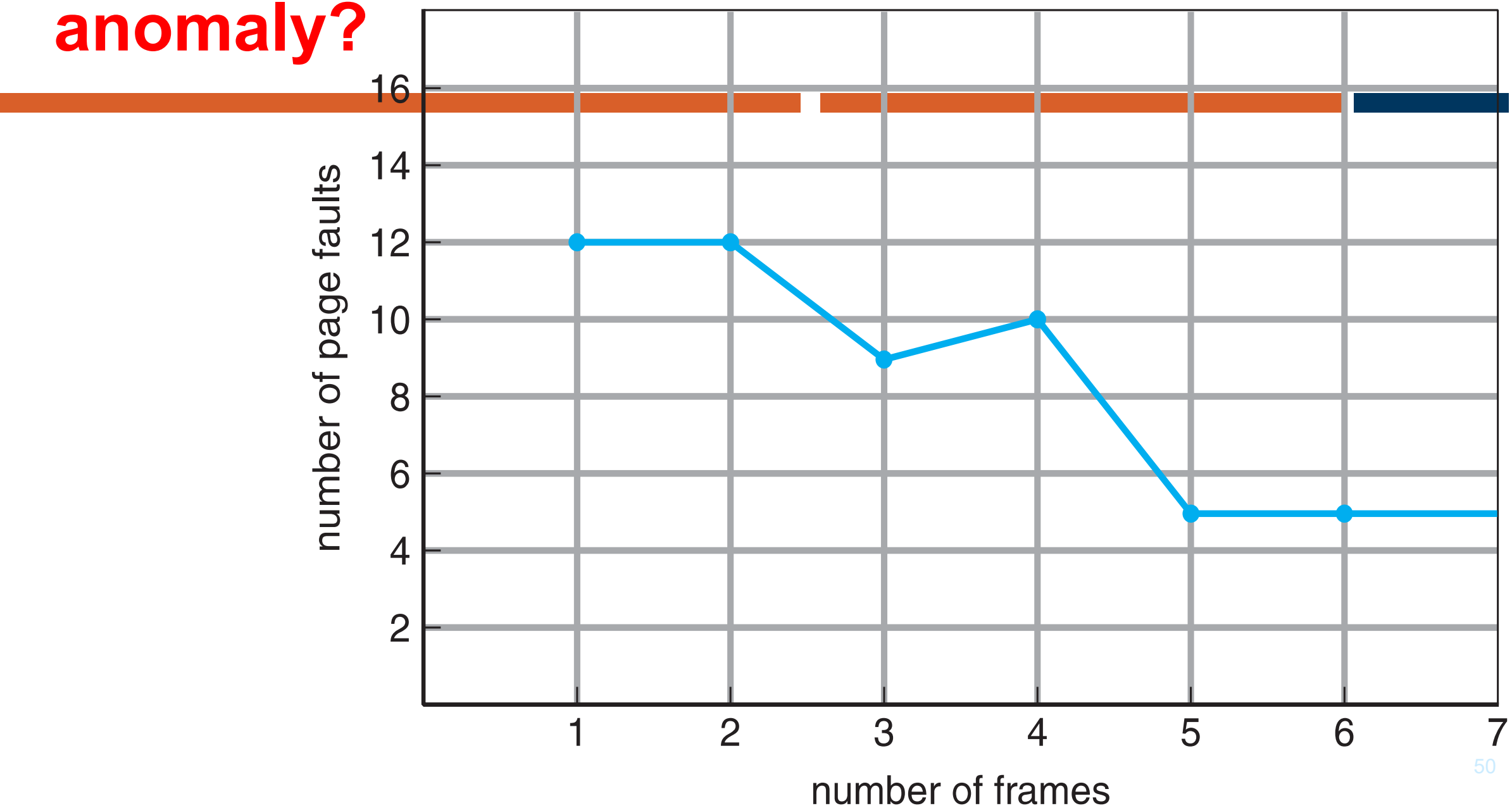
Case-02: When frame size = 4



Number of page faults = 8


16
14
12
10
8
6
4
2
number of page faults
1
2
3
4
5
6
7
number of frames

Do we always experience Belady's anomaly?



Belady's Anomaly

- Phenomenon where increasing the number of page frames results in an increase in page faults.
- *Belady's anomaly* is commonly experienced with:
  - FIFO.
- But – not LRU, OPT, and variants.
- Why?

Stack Algorithms

- LRU is an example of a stack algorithm.
- ***Stack algorithms do not suffer from Belady's anomaly – Why?***

Stack Algorithms

- **Stack algorithm:** Set of pages in memory for N frames is *always a subset* of the set of pages that would be in memory with N + 1 frames.

- Increasing memory size is guaranteed to reduce (or at least not increase) the number of page faults.

- Stack algorithms (e.g., LRU), fit with (page) memory access patterns – so: +1 frame will never cause more page faults.

LRU vs OPT vs FIFO

- Who wins?

LRU vs OPT vs FIFO

- (1) OPT, (2) LRU, (3) FIFO.
- LRU and OPT are cases of *stack algorithms* that don't suffer from *Belady's Anomaly*.
- But – there are more – e.g. Clock algorithm.

Clock: Approximate LRU

- Page X referenced – *In memory* –
  (Ref Bit/Used bit = 1)

- Page X referenced – *Not in memory* (page fault):

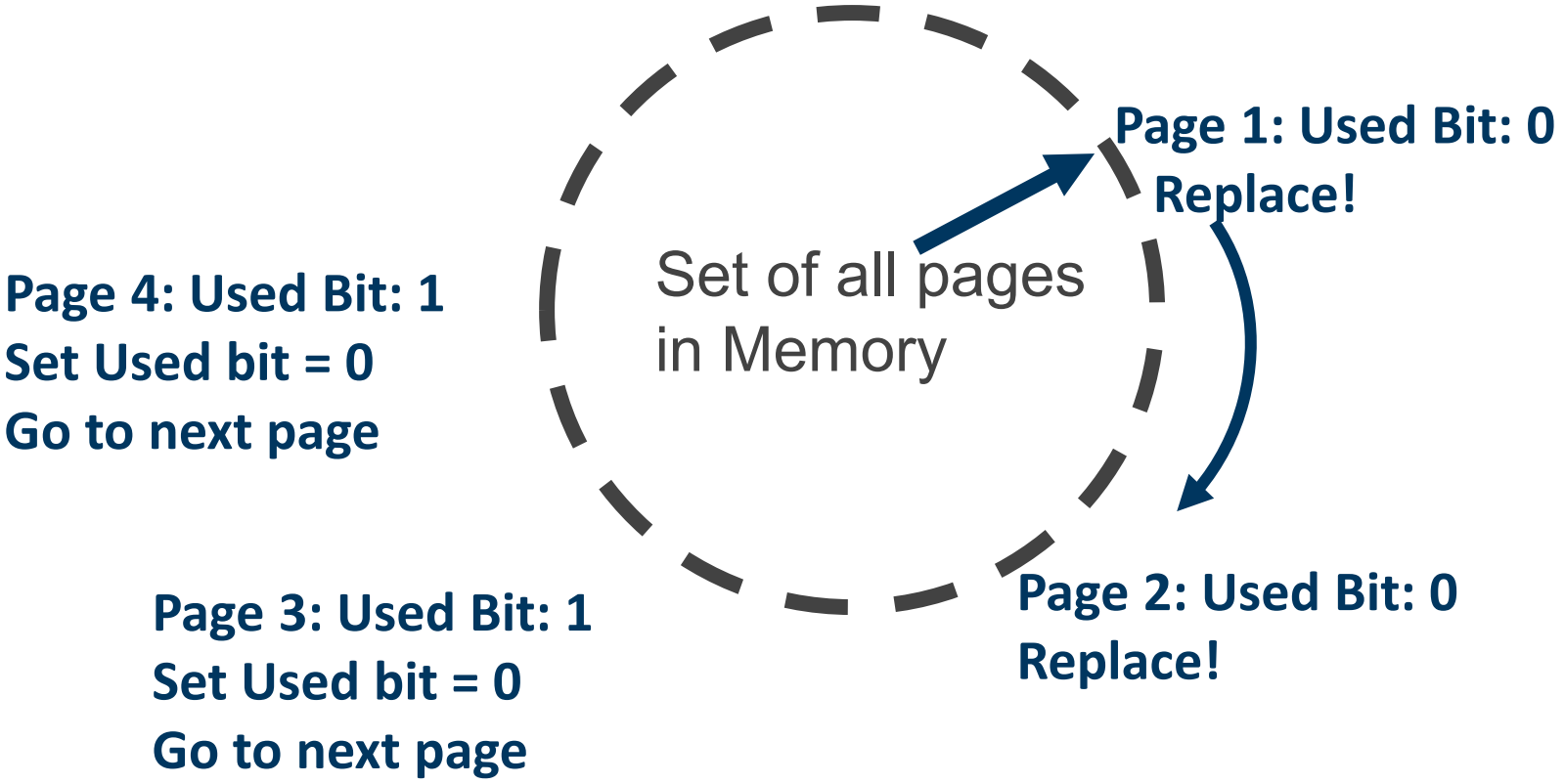
  - Look at page pointed to by clock hand
    – check *Ref Bit* (*Used Bit*) – Increment clock hand.

  - **IF Ref Bit = 0 :** Set bit = 1 : Replace page.
  - **IF Ref Bit = 1 :** Set bit = 0.

Clock: Approximate LRU

- Reference (used) bit set whenever a page is referenced.
- Advance clock-hand whenever there is a page-fault.
- So – clock-hand sweeps over pages looking for page to replace (with ***ref bit/used bit = 0***).
- So – replace pages that haven't been referenced for one complete revolution.

Clock: Approximate LRU



```
func Clock_Replacement
begin
   while (victim page not found) do
      if (used bit for current page = 0) then
         replace current page
      else
         reset used bit
      end if
      advance clock pointer
   end while
end Clock_Replacement
```

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**
- **Initially:** Assume that the first 3 frames are all filled: {0, 4, 1}. (total page faults = 3).
- **Ref bits:** {0, 0, 0}, assume ref bits cleared for this starting point.

- Page 4 (ref bit set = 1) , clock-hand on 1st frame element, (no page fault).

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-4:** Frame = **{ 0 , 4 , 1}**, ref bits = **{ 0 , 1 , 0 }**

- Page 4 (**ref bit set = 1**) , clock-hand on 1st frame element, (no page fault).

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-5:** Frame = **{ 2, 4 , 1 }**, ref bits = **{ 1, 1 , 0 }**

- Page 0 replaced (ref bit **=0**, so replaced!, clock-hand on 2nd frame element, (total page faults = 4).

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-6:** Frame = **{ 2 , 4 , 1 },** ref bits = **{ 1 , 1, 0 },** clock-hand on 2nd frame element, (no page fault)

- Page 4 (ref bit set = **1**), already in memory, clock-hand on 2nd frame element, (no page fault).

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-7:** Frame **= { 2 , 4 , 3 }**, Ref bits = **{ 1, 0 , 1 },** Page 4 stays, was checked (ref bit set **= 0**), clock-hand moves to frame 3 then frame 1 (ref bit =0, replace page 1 in frame 3).

- Page 3 (ref bit **= 1**), now in frame 3, (total page faults = 5).

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-8:** Frame = **{ 2 , 4, 3 }**, Ref bits = **{ 1, 1 , 1 }**, Page 4 referenced again (but already in memory – no page fault), Page 4 **ref bit set =1**.

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-9:** Frame = **{ 2 , 4, 3 }**, Ref bits = **{ 1 , 1 , 1 }**, Page 2 referenced again, Page 2 **ref bit = 1**, (already in memory – no page fault).

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-10:** Frame = **{ 2 , 4 , 3 }**, Ref bits = **{ 1 , 1 , 1 }**, Page 4 referenced again, Page 4 **ref bit = 1**, (already in memory – no page fault).

Clock Algorithm – Example

- Reference string :

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-11:** Frame = **{ ?, ? , ? }**, Ref bits = **{ ? , ? , ? }**

Clock Algorithm – Example

- Reference string :

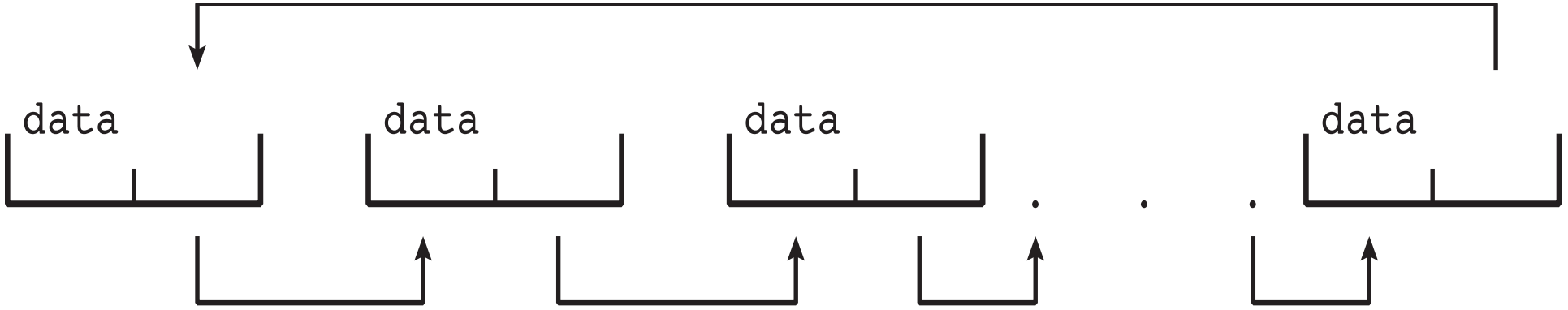
0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

- Frames: **3**

- **Pass-11:** Frame = **{ 0 , 4 , 3 }**, Ref bits = **{ 1 , 0 , 0 }**, Page 0 referenced, frame 1 gets 2nd chance (Ref bit =1), advance clock-pointer, frame 2 gets 2nd chance (Ref bit =1), frame 3 gets 2nd chance, advance clock pointer, replace page 2 in frame 1 (Ref bit =0), pf=6.

Clock Algorithm

- Pages kept in circular linked list (one reference bit per page).



- Advantages? – Disadvantages?

Clock Algorithm

- ***Ref Bit/Used Bit*** only indicates if page was used at all since last checked (not how long exactly).

- Simple hardware requirements – fast, since setting a single bit on each memory access.

- Page checking is faster – we only search the pages until we find one with a **‘0’** reference bit.

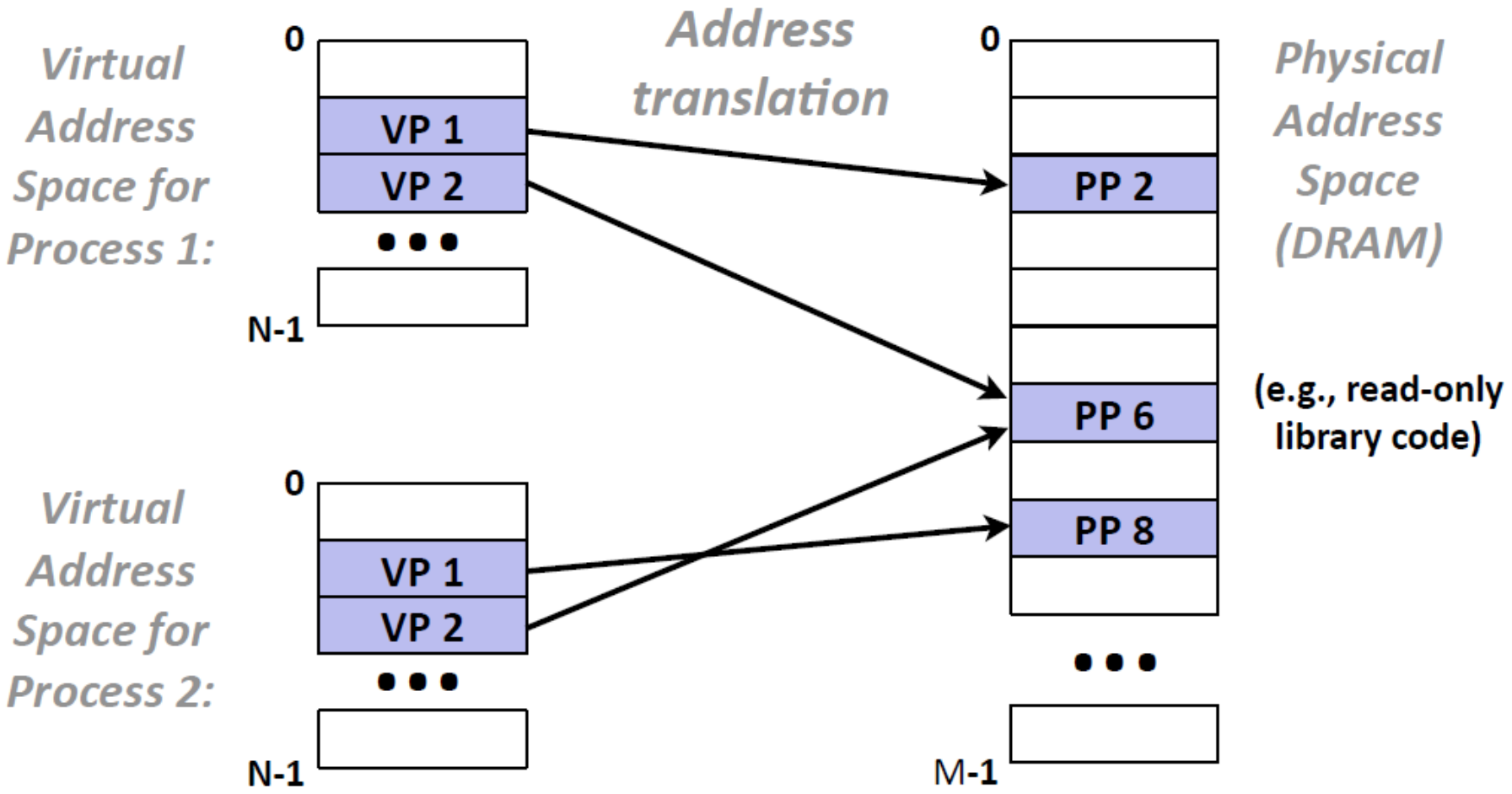
Clock Algorithm: Issues?

- Always find a page to replace.
- Worst case – all use bits set – just loops around.
- Algorithm performance in this case?

Clock Algorithm: Issues?

- Worst case – all use bits set – just loops around.
- Algorithm performance in this case: **O(N)** for N pages.
- ***What else?***

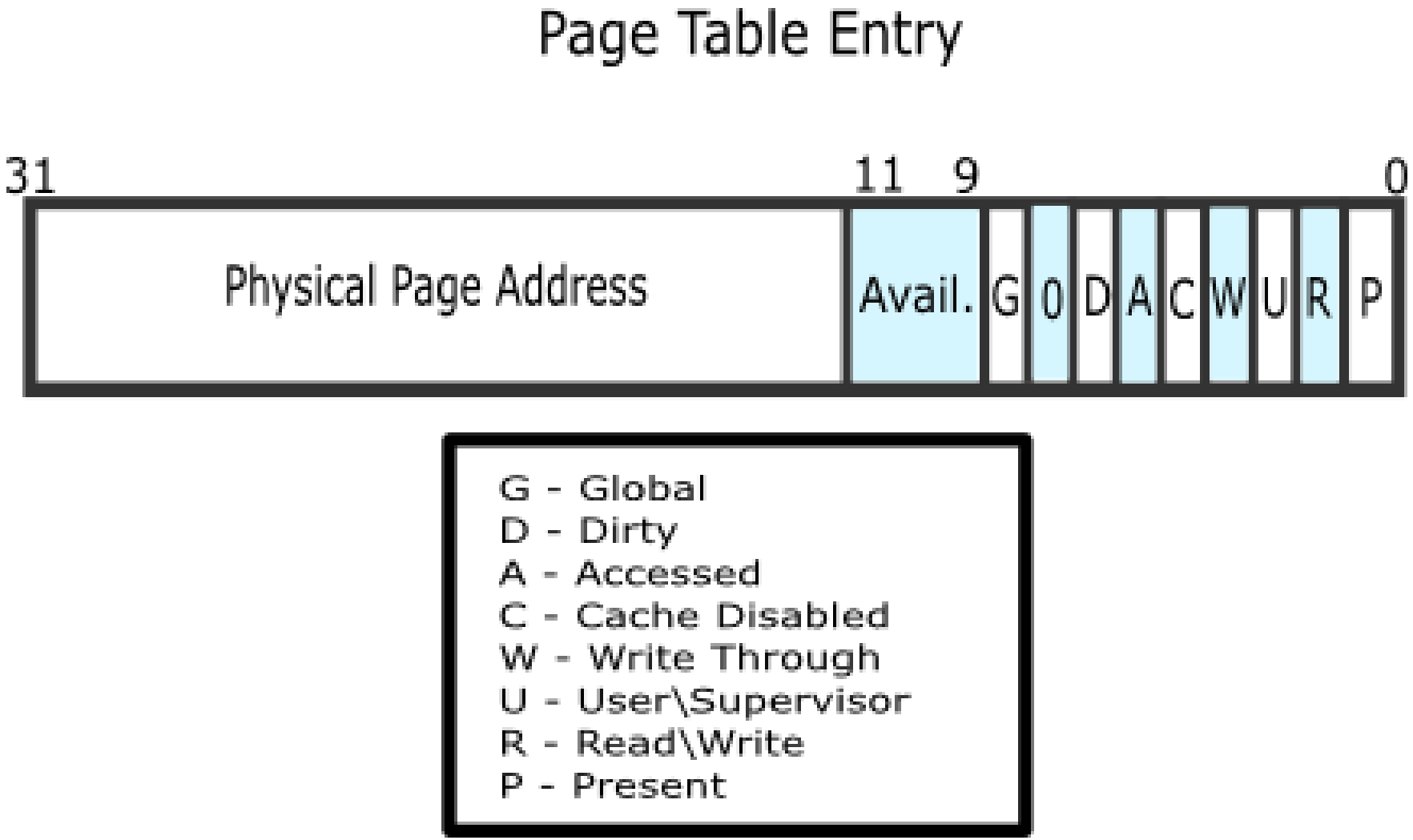
Shared Frames & Dirty Pages!



Dirty Bits and Pages

- ***Dirty bit***: one bit for each page frame, set by hardware whenever the page is modified.
- ***Dirty page***: Pages in the main (or cache) memory modified after being swapped in – if changed, must be written to disk before its page frame is reused.
- **Dirty bit =0:** Page changed but written back to disk.
- **Dirty bit =1:** Page changed but NOT written back to disk.

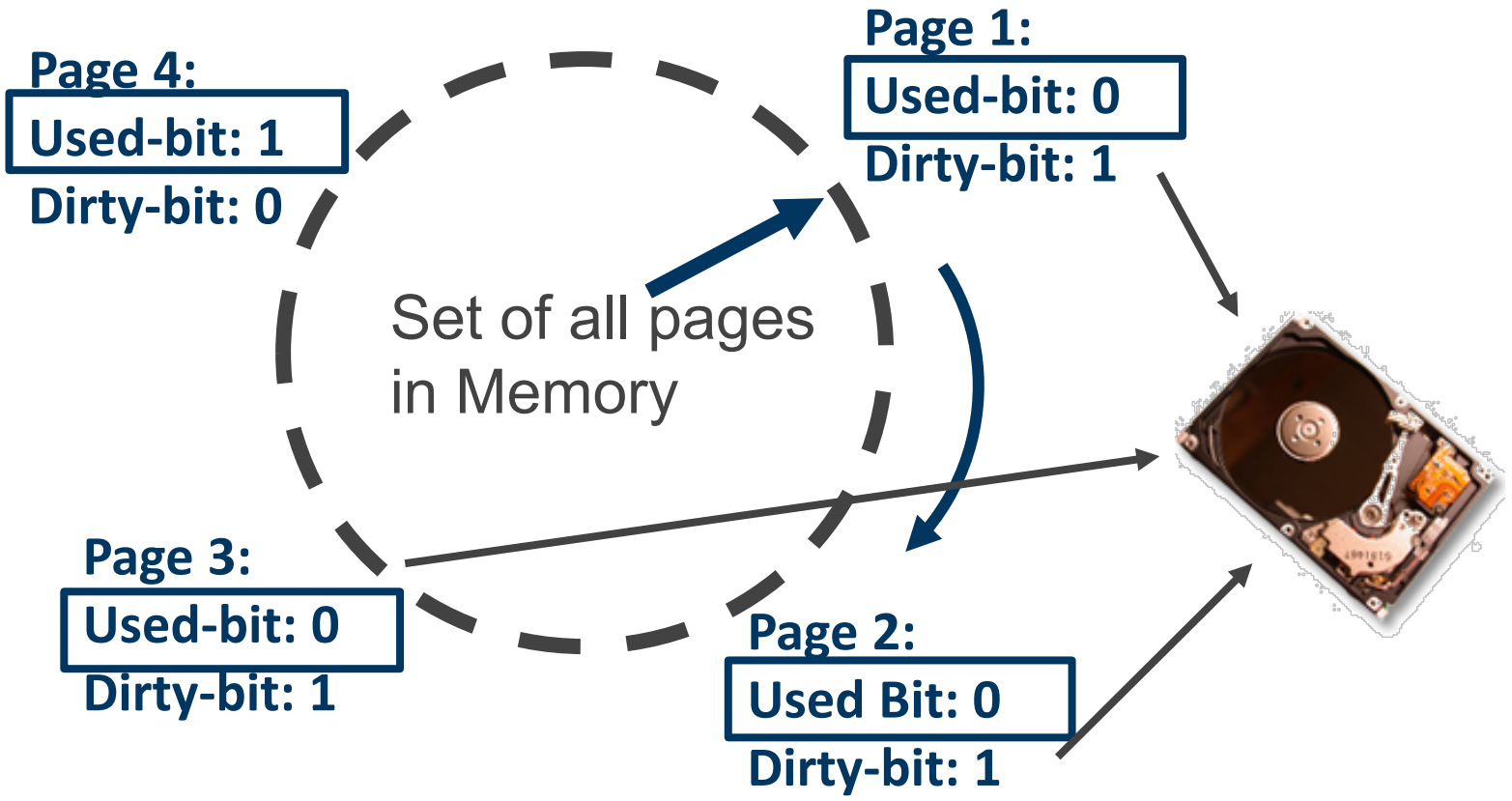
Page Table Entry



Dirty Bits and Pages

- *Dirty bit*: one bit for each page frame (in PTE) – ***Dirty bit = 1*** when page is modified.
- Must account for dirty pages – writing pages to disk before their physical frames are reused.
- *Modify clock algorithm to allow dirty pages to always survive one sweep of the clock-hand*.

Second Chance (Clock)



```
func Clock_Replacement
begin
   while (victim page not found) do
      if (used bit for current page = 0) then
         replace current page
      else
         reset used bit
      end if
      advance clock pointer
   end while
end Clock_Replacement
```

(used = 0, dirty = 0)

Use (used, dirty) table

Second Chance Algorithm

Before clock sweep

used	*dirty*
0	0
0	1
1	0
1	1

After clock sweep

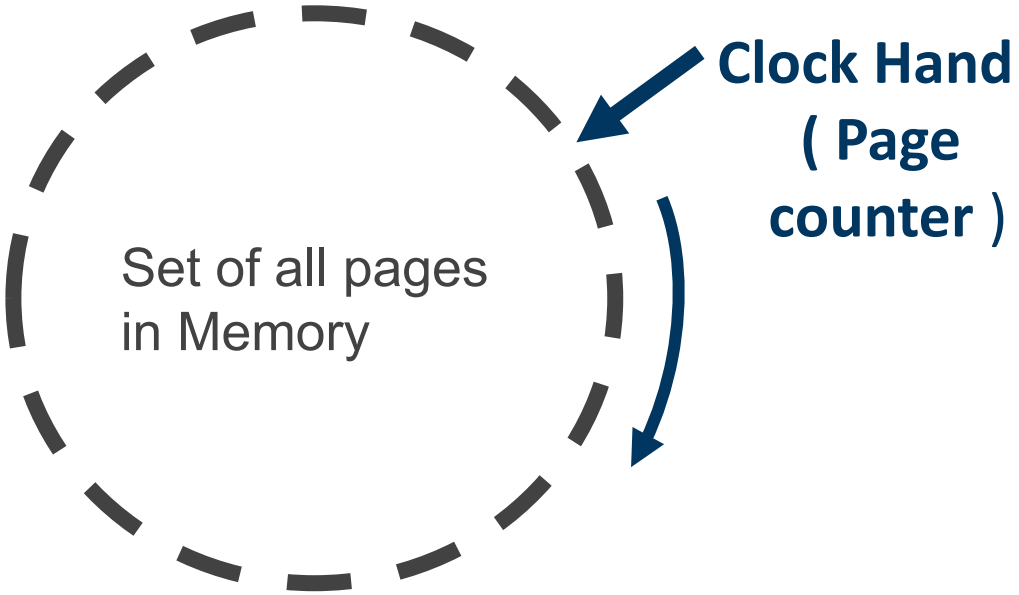
used	*dirty*
replace page	
0	0
0	0
0	1

Yet Another Chance Clock

- What if we don't want to replace heavily used pages?
- Can we give these pages more of a chance to stay in main memory even longer?

N Chance Clock

- Give each page N chances before selecting as a page replacement victim.



N Chance Clock Algorithm

- On page fault – checks **used-bit**:
  - **0 → If ref bit is off (0), page has been used N times – replace page!**
  - **1 → increment counter; if count=N (clear used-bit).**
- Means that clock hand has to sweep by N times before page is replaced.

N Chance Clock: N = ?

- **Larger N?**
  - ???

N Chance Clock: N = ?

- **Larger N?** Better approximation of LRU.
  - If N ~ 1000, really good approximation.
  - For large N algorithm cycles to find an eligible victim page.

N Chance Clock: N = ?

- **Smaller N?**
  - ???

N Chance Clock: N = ?

- **Smaller N?** More efficient.
  - Pages replaced more often.
  - Infrequently used pages not kept for too long.
  - Larger N accounts for these infrequent accesses.

Page Replacement Algorithms

- **FIFO:** Place pages in queue, replace end page.
- **LRU:** Replace least recently used page.
- **Clock:** Approximation to LRU
  - Arrange all pages in circular linked list.
  - Sweep through them, marking as not “in use”
  - If page not “in use” for N passes, then replace.

What OS uses what?

- All editions of Linux / Unix (MacOS) / Windows NT up to Windows 11:
  - All use various approximations of LRU.:
  - N chance Clock.
  - Aging (NFU): LRU variant – oldest page in last X pages replaced – keeps pages referenced recently, but less frequently – rather than more frequently referenced pages further in past.

Summary: Page Replacement

- **Experimental Analysis:** All algorithms perform badly if processes have insufficient physical memory.
- **All approaches → optimal:** As physical memory allocated to processes approaches virtual memory size.
- **More processes running concurrently:** Less physical memory each process can have.
- **Critical OS issue:** *Must decide how many processes (and frames per process) concurrently share memory.*